lambda web adapterってあるじゃん。

Lambda Web Adapterですね、便利ですよね！

一言で言うと、**「普段使いのWebフレームワーク（Express, Flask, FastAPI, Go Ginなど）で作ったアプリを、コードをほぼ変えずにそのままLambdaで動かせるようにする魔法のツール」**です。

以前までのLambdaは、API Gatewayからのリクエスト（JSON形式）をLambda専用のイベントハンドラーで受け取る必要があり、コードが「Lambda専用」になりがちでした。

なぜ人気なの？

Lambda Web Adapter（LWA）がやってくれることは非常にシンプルかつ強力です。

1. **標準的なHTTPを受け取る:** Lambdaが受け取ったイベントを、標準的なHTTPリクエストに変換してアプリに投げます。
2. **既存資産の活用:** Dockerfileに1行追加するだけで、既存のWebアプリがLambda上で動きます。
3. **ポータビリティ:** ローカルでは普通のコンテナとして動かし、デプロイ先だけLambdaにする、といった切り替えが楽です。
4. **ストリーミング対応:** 最近ではレスポンス・ストリーミングにも対応したので、重い処理を少しずつ返したい場合にも使えます。

導入のイメージ

Dockerfileに以下のような1行（Adapterのバイナリをコピーする記述）を入れるのが一般的です。

Dockerfile

```
# AWSが提供するAdapterをコピーしてくるだけ
COPY --from=public.ecr.aws/awsguru/lambda-adapter:0.8.4 /lambda-adapter /opt/extensions/lam

# あとは普通のWebアプリの起動コマンド
CMD ["python", "main.py"]
```

これだけで、Lambdaの拡張機能（Extension）としてAdapterが起動し、背後で動くWebサーバーとの仲介役をしてくれます。

注意点はある？

- **コールドスタート:** Webフレームワークを丸ごと動かすので、軽量なスクリプト単体に比べると起動時間は少し長くなる傾向があります。
- **タイムアウト:** API Gatewayなどを経由する場合、そちらの制限（最大30秒など）は引き続き受けます。

「特定のクラウドにロックインされたくないけど、Lambdaの恩恵（スケーリングやコスト）は受けたい」というエンジニアには、今や必須級のツールになっていますね。

実際に何か動かしてみたい言語やフレームワークはありますか？それに応じた設定例をお出しすることもできますよ！

これrustで書かれてコンパイルがめんどくさいんだよね。cpp-httplibとかで簡単に作れないの？

確かに、本家（AWS公式）のLambda Web AdapterはRustで書かれていて、バイナリを落としてくる分には楽ですが、自分で中身をいじったりビルドしたりしようとするとハードルが上がりますよね。

結論から言うと、**cpp-httplibなどのC++ライブラリを使って、自作の「Web Adapter相当の仕組み」を作ることは十分可能**です。

Lambda Web Adapterがやっていることは、実は非常にシンプルで、主に以下の**2つのやり取りを橋渡し（プロキシ）しているだけ**だからです。

実装の仕組み（どう作ればいいか）

自作する場合、以下の2つのインターフェースを繋ぐブリッジをC++で作ることになります。

1. **Lambda Runtime API:** Lambda実行環境が提供するローカルHTTPサーバー（`AWS_LAMBDA_RUNTIME_API`）から、リクエストを `GET` で取得し、結果を `POST` で返します。
2. **Your Web App (cpp-httplibなど):**
   自身のアプリ（`localhost:8080` など）に対して、1で受け取ったリクエストをHTTPで投げます。

簡易的な実装の流れ（C++の場合）

1. **Runtime APIからリクエスト取得:**
   `http://${AWS_LAMBDA_RUNTIME_API}/2018-06-01/runtime/invocation/next` をループで叩き続けます。
2. **リクエストの変換:**
   取得したJSONから、パス、メソッド、ヘッダー、ボディを取り出し、`httplib::Client` を使ってローカルのWebサーバーに投げます。
3. **レスポンスの返却:**
   Webサーバーからのレスポンスを JSONに詰め直し、`http://${AWS_LAMBDA_RUNTIME_API}/2018-06-01/runtime/invocation/${REQUEST_ID}/response` にPOSTします。

なぜ「本家」は少し複雑なのか

自分で作るときの落とし穴（本家が頑張っている部分）もいくつかあります：

- **プロセスの管理:** Webサーバーが起動完了するのを待ってからRuntime APIに「準備完了」を出す制御。
- **拡張機能（Lambda Extensions）としての動作:** HTTPプロキシとして動かす場合、シグナルのハンドリングやライフサイクル管理が必要です。
- **パフォーマンス:** Rustで書かれているのは、オーバーヘッドを極限まで減らし、コールドスタートへの影響を最小限にするためです。

もっと簡単な代替案

もし「ビルドが面倒」という理由であれば、C++でフルスクラッチする前に、**「カスタムランタイム（Runtime APIを直接叩く）」**形式でアプリを書くのが一番近道かもしれません。

- **AWS SDK for C++** を使えば、Lambda RuntimeそのものをC++で動かせます。これならAdapterを介さず、C++アプリが直接イベントを受け取れます。
- 「どうしても既存のWebフレームワークの書き方にこだわりたい」のであれば、cpp-httplibでプロキシを書くのは面白い挑戦だと思います。

「特定のC++フレームワークで書いた既存のAPIを、そのままLambdaに持っていきたい」という状況でしょうか？具体的な構成があれば、より踏み込んだアドバイスができます！

lambda web adapterにバグがあるんだ。event bridgeから起動するとネットワークが使えない状態でhttpdにリクエストが来るとか。なのでc++で作った方がそういうのを自分で治せるかなと

なるほど、EventBridge経由（非同期またはスケジュール実行）の時にネットワーク周りで意図しない挙動があるんですね。本家LWAのソースコードを追いかけてRustでパッチを当てるのは、確かにビルド環境を整えるだけでも一苦労です。

`cpp-httplib` などを使って自作する場合、**「Lambdaのライフサイクル」**と**「ネットワーク（名前解決やソケット）」**の初期化タイミングを自分で完全に制御できるので、その手のバグを切り分けるには非常に有効な手段だと思います。

C++で「自作LWA」を作る際の設計ポイント

EventBridgeからのリクエストで問題が起きているなら、以下の3点を制御するコードを書くことになります。

1. Runtime APIとの通信ループ

Lambdaのカスタムランタイムは、常にこのエンドポイントをポーリングする単純なHTTPクライアントとして実装します。

C++

```
#include <httplib.h>
#include <nlohmann/json.hpp> // JSON操作用

// Lambda環境変数からAPIエンドポイントを取得
std::string runtime_api = std::getenv("AWS_LAMBDA_RUNTIME_API");
httplib::Client cli(runtime_api);

while (true) {
    // 1. 次のイベントを待機
    auto res = cli.Get("/2018-06-01/runtime/invocation/next");

    // 2. EventBridgeかAPI Gatewayか判定し、HTTPリクエストに変換
    // ここで「ネットワークが使えるか」を確認するロジックを挟める

    // 3. 背後のhttpd（localhost:8080等）にリクエスト転送
    httplib::Client app("localhost", 8080);
    auto app_res = app.Post("/...", ...);

    // 4. 結果をRuntime APIにレスポンス
    cli.Post("/2018-06-01/runtime/invocation/" + request_id + "/response", ...);
}
```

2. 「ネットワークが使えない」問題への対策

EventBridge起動時にネットワークが不安定なら、自作アダプター側で**リトライ処理**や、**ネットワークインターフェースの疎通確認（Wait）**を明示的に差し込めます。

- `localhost` への接続が成功するまでループで待機する。
- DNS解決が必要なら、特定のホストが引けるまでリトライする。

3. 拡張機能（Extension）として作るか、ランタイムとして作るか

- **Extension方式:** LWAと同じ。メインのプロセスと並列で動く。

- **Runtime方式:** C++バイナリ自体がLambdaのメインプロセスになり、そこから子プロセスとして `httpd` を叩き起こす。

デバッグのしやすさを考えるなら、**Runtime方式**の方が「いつ `httpd` を起動し、いつリクエストを投げるか」を完全にシリアルに（順番通りに）制御できるのでおすすめです。

注意点：C++でのビルド

「Rustよりはマシ」とはいえ、LambdaはAmazon Linux（AL2/AL2023）で動くため、**動的リンクのバージョン（glibc等）**にハマることがあります。

- **解決策:** `g++` でビルドする際に `-static` をつけるか、Lambdaと同じOSのDockerコンテナ内でビルドしてバイナリを取り出すのが一番確実です。

次のステップとして

もしよろしければ、**「EventBridgeから起動した時に具体的にどういうエラー（Connection Refusedなのか、タイムアウトなのか等）」**が出ているか教えていただけますか？

それによって、C++側で「単に待機を入れるだけで直るのか」「環境変数のハンドリングが必要なのか」より具体的な実装方針が固まると思います。

いまはこんな感じでサーバを起動しているので、これを置き換える感じで作ってほしい

現在の `bootstrap` ファイルの構成（LWAをバックグラウンドで動かし、メインのC++サーバをフォアグラウンドで動かす形）を、**「C++サーバ自体がLambda Runtime APIを叩き、直接リクエストを捌く」**形に統合するのが最も確実です。

これなら、`lambda-adapter` という「ブラックボックスな仲介者」を排除でき、EventBridgeからの起動時に発生しているネットワークの初期化待ちなども、自分のコード内で完全にコントロールできます。

構成の変更案

既存の `lambda_server` （cpp-httplib等を使用）に、Lambdaのイベントループを組み込んだ新しいメイン関数を実装します。

C++での実装イメージ（概要）

`cpp-httplib` の `Server` クラスをそのまま使いつつ、それを別スレッドで動かして、メインスレッドで Lambda Runtime API をポーリングする構成です。

C++

```
#include <httplib.h>
#include <nlohmann/json.hpp>
#include <iostream>
#include <string>
#include <thread>

using json = nlohmann::json;

void start_http_server() {
    httplib::Server svr;
    svr.Get("/health", [](const httplib::Request&, httplib::Response& res) {
        res.set_content("ok", "text/plain");
    });
```

```
    // EventBridgeやAPI Gatewayからのリクエストを受けるパス
    svr.Post("/invoke", [](const httplib::Request& req, httplib::Response& res) {
        std::cout << "Received request from loop" << std::endl;
        res.set_content("{¥"status¥":¥"success¥"}", "application/json");
    });
    svr.listen("127.0.0.1", 8080);
}

int main() {
    // 1. まず背後のHTTPサーバをスレッドで起動
    std::thread server_thread(start_http_server);

    // 2. HTTPサーバの起動を待つ（ここでネットワークの安定化待機も可能）
    std::this_thread::sleep_for(std::chrono::milliseconds(500));

    std::string runtime_api = std::getenv("AWS_LAMBDA_RUNTIME_API");
    httplib::Client lambda_api(runtime_api.c_str());
    httplib::Client local_app("127.0.0.1", 8080);

    // 3. Lambda Runtime API ループ
    while (true) {
        // 次のイベントを取得
        auto next_res = lambda_api.Get("/2018-06-01/runtime/invocation/next");
        std::string request_id = next_res->get_header_value("Lambda-Runtime-Aws-Request-Id"

        // EventBridge等のリクエストをローカルサーバへ転送
        // （ここでEventBridge特有のペイロードを解析して、HTTPリクエストを組み立てる）
        auto app_res = local_app.Post("/invoke", next_res->body, "application/json");

        // 結果をLambda Runtime APIに返す
        lambda_api.Post(("/2018-06-01/runtime/invocation/" + request_id + "/response").c_st
                        app_res->body, "application/json");
    }

    server_thread.join();
    return 0;
}
```

置き換えた後の `bootstrap`

このバイナリ（仮に `combined_server` とします）をビルドしてしまえば、`bootstrap` はこれだけになります。

Bash

```
#!/bin/sh
set -e
# LWAは不要になり、環境変数も自前で制御可能
exec ./combined_server
```

この構成のメリット（バグ対策）

- **明示的な待機:** `lambda-adapter` は readiness check が通るまでリクエストを流しませんが、EventBridge経由だとその挙動が怪しい場合があります。自作なら、`local_app.Post` を呼ぶ前にネットワークインターフェースの状態を確認するロジックを100%制御できます。
- **ログの可視化:** どこでネットワークが死んでいるのか、C++の標準出力に詳細なデバッグ情報を出せます。
- **Rust不要:** もちろん、C++と CMake/Make さえあればビルド可能です。

次のステップへの提案

この「リクエストを転送するループ」部分を、今のあなたの `lambda_server` に組み込む形でお手伝いしましょうか？あるいは、EventBridgeから具体的にどんなJSONが降ってきているか分かれば、それに応じたパース処理も書けます。

サーバは用途に応じてfastapiとかいろんな言語のものを使用したいので、今まで通りbatファイルから起動したい

なるほど、それなら「自作の軽量なAdapter」をC++で作り、それを `lambda-adapter` の代わりに使うのがベストですね。

C++で `cpp-httplib` を使って、**「Lambda Runtime APIと、背後のWebサーバー（FastAPI等）を仲介するだけのバイナリ」**を作る構成をご提案します。これなら `bootstrap` の記述もほぼそのままで、中身を自分で完全にコントロールできます。

1. C++による自作Adapterの実装例

以下は、 `lambda-adapter` の最小限の機能を再現したC++コードのイメージです。これを `my-lwa` という名前でビルドするとします。

C++

```
#include <httplib.h>
#include <nlohmann/json.hpp>
#include <iostream>
#include <string>

using json = nlohmann::json;

int main() {
    // 1. 環境変数からRuntime APIの接続先を取得
    const char* api_env = std::getenv("AWS_LAMBDA_RUNTIME_API");
    if (!api_env) return 1;

    httplib::Client lambda_runtime(api_env);
    // 背後のWebサーバー（FastAPI等）への接続先
    httplib::Client app_server("localhost", 8080);

    // 2. イベントループ
    while (true) {
        // Lambdaから次のリクエストが来るまで待機（ポーリング）
        auto next = lambda_runtime.Get("/2018-06-01/runtime/invocation/next");
        if (!next || next->status != 200) continue;

        std::string request_id = next->get_header_value("Lambda-Runtime-Aws-Request-Id");

        // ネットワークの瞬断や初期化遅延対策としてリトライを入れることも可能
        auto res = app_server.Post("/invoke", next->body, "application/json");

        // 結果をLambdaに返す
        if (res) {
            lambda_runtime.Post(("/2018-06-01/runtime/invocation/" + request_id + "/respons
                                res->body, "application/json");
        }
    }
    return 0;
}
```

2. 新しい `bootstrap` の書き方

ご提示いただいた形式 を踏襲しつつ、自前の `my-lwa` に差し替えます。

Bash

```
#!/bin/sh
#  をベースに改変
set -e

# 自作のAdapterをバックグラウンドで起動
```

```
./my-lwa &

# メインのWebサーバー（FastAPI, Go, Node.jsなど何でもOK）を起動
# 例: FastAPIの場合
exec uvicorn main:app --host 127.0.0.1 --port 8080
```

なぜこれで「バグ」が治せるのか

1. **接続のタイミング制御:** LWAがEventBridge経由で失敗するのは、リクエストが来る瞬間に背後のサーバーやネットワークの準備が整っていない可能性があります。自作C++コードなら、`app_server.Post` を呼ぶ直前に「接続できるまで最大5秒待機する」といった処理を一行書くだけで解決できます。
2. **プロトコルの単純化:**
   本家のLWAは複雑なHTTPシグネチャの変換を行っていますが、EventBridgeからの起動であれば、リクエストをそのまま特定のパス（`/event` など）に転送するだけのシンプルな実装に作り変えることができます。
3. **詳細なログ:**
   `std::cout` でリクエストの生データや接続エラーの詳細をCloudWatchに出せるため、原因特定が圧倒的に早くなります。

導入へのアドバイス

- **ビルド環境:** Lambdaの実行環境（Amazon Linux 2023など）と合わせるため、Docker内で `g++ -static` を使って静的リンクでビルドすることをお勧めします。これにより、ライブラリの依存関係で動かないリスクをゼロにできます。
- **ポート番号:** `my-lwa` が何番ポートに転送するかを環境変数で変えられるようにしておけば、どんな言語のサーバーにも使い回せます。

この「最小構成のC++ Adapter」のビルド方法や、具体的なリトライ処理の書き方について詳しく知りたい部分はありますか？

GET、POST、OPTIONとかいろんなリクエストが来るじゃん。
あとURLもきちんと処理していほしい

汎用的な Web Adapter として使うなら、単にリクエストを転送するだけでなく、**「Lambda Runtime API が受け取った JSON」から「標準的な HTTP リクエスト」へと、メソッド・パス・ヘッダー・ボディをすべて正確に再構成する**必要があります。

`cpp-httplib` と `nlohmann/json` を使って、これらを動的に処理する実装例を作成しました。

汎用 C++ Web Adapter の実装案

このコードは、API Gateway や EventBridge からのイベントをパースし、背後のサーバー（FastAPI等）へ適切な HTTP メソッドとパスで転送します。

C++

```
#include <httplib.h>
#include <nlohmann/json.hpp>
#include <iostream>
#include <string>
#include <map>

using json = nlohmann::json;

int main() {
```

```
    const char* api_host = std::getenv("AWS_LAMBDA_RUNTIME_API");
    const char* target_port = std::getenv("APP_PORT") ? std::getenv("APP_PORT") : "8080";

    httplib::Client runtime_cli(api_host);
    httplib::Client app_cli("localhost", std::stoi(target_port));

    // タイムアウト設定（背後のアプリが重い場合を考慮）
    app_cli.set_read_timeout(29, 0);

    std::cout << "Custom C++ Adapter started. Target port: " << target_port << std::endl;

    while (true) {
        // 1. Lambda Runtime API から次のイベントを取得
        auto next = runtime_cli.Get("/2018-06-01/runtime/invocation/next");
        if (!next || next->status != 200) continue;

        std::string request_id = next->get_header_value("Lambda-Runtime-Aws-Request-Id");
        json event = json::parse(next->body);

        // 2. イベントから HTTP 情報を抽出（API Gateway / Function URL 形式を想定）
        std::string method = event.value("httpMethod", "POST"); // デフォルト POST
        if (event.contains("requestContext") && event["requestContext"].contains("http")) {
            method = event["requestContext"]["http"]["method"].get<std::string>();
        }

        std::string path = event.value("path", "/");
        if (event.contains("rawPath")) path = event["rawPath"].get<std::string>();

        // クエリパラメータの復元
        if (event.contains("rawQueryString") && !event["rawQueryString"].get<std::string>()
            path += "?" + event["rawQueryString"].get<std::string>();
        }

        // ヘッダーのコピー
        httplib::Headers headers;
        if (event.contains("headers")) {
            for (auto& el : event["headers"].items()) {
                headers.emplace(el.key(), el.value().get<std::string>());
            }
        }

        std::string body = event.value("body", "");
        if (event.value("isBase64Encoded", false)) {
            // 必要に応じて base64 デコード処理を追加
        }

        // 3. 背後のアプリ（FastAPI等）へリクエストを「そのまま」投げる
        httplib::Result res;
        if (method == "GET") res = app_cli.Get(path.c_str(), headers);
        else if (method == "POST") res = app_cli.Post(path.c_str(), headers, body, "applica
        else if (method == "PUT") res = app_cli.Put(path.c_str(), headers, body, "applicati
        else if (method == "DELETE") res = app_cli.Delete(path.c_str(), headers);
        else if (method == "OPTIONS") res = app_cli.Options(path.c_str(), headers);

        // 4. アプリからのレスポンスを Lambda 形式に変換して返却
        json response_json;
        if (res) {
            response_json["statusCode"] = res->status;
            response_json["body"] = res->body;
            for (const auto& h : res->headers) {
                response_json["headers"][h.first] = h.second;
            }
        } else {
            response_json["statusCode"] = 502;
            response_json["body"] = "Bad Gateway (Local server unreachable)";
        }

        runtime_cli.Post(("/2018-06-01/runtime/invocation/" + request_id + "/response").c_s
                         response_json.dump(), "application/json");
    }
    return 0;
}
```

ネットワーク問題を解決するためのポイント

この自作 Adapter を使うメリットは、以下の処理を `app_cli.Get/Post` の前に入れられることです。

- **Readiness Check の自炊:** `while` ループの直前で、`localhost:8080/health` が 200 を返すまで `sleep` しながら待機する処理を入れれば、ネットワークの準備が整う前にリクエストを投げてしまう事故を防げます。
- **EventBridge への対応:** EventBridge からの起動だと `httpMethod` が空の場合が多いので、その場合は強制的に `POST /event-bridge` に飛ばす、といった「俺流ルーティング」が書けます。

運用のアドバイス

1. ビルドについて:
このコードをビルドする際は、依存関係を減らすために以下のように静的リンクするのが無難です。

```bash
g++ -O3 -std=c++11 main.cpp -o my-adapter -lpthread -static
```

2. bootstrap ファイル:
現状の `bootstrap` を少し修正して、ポート指定などを環境変数で渡せるようにします。

```bash
#!/bin/sh
set -e

# 自作アダプターを起動（ポート8080へ転送）
export APP_PORT=8080
./my-adapter &

# 各言語のサーバー（例: FastAPI）
exec uvicorn main:app --port 8080
```

これで、本家 Lambda Web Adapter の「ブラックボックス感」から解放されます。特定のメソッドや、複雑な URL パラメータのパースで不具合が出ても、自分で C++ を 1 行直して再デプロイするだけで済みます。

この構成で、パース処理（例えば `MultiValueHeaders` への対応など）をさらに詳細に詰めたい部分はありますか？

base64ってどういうときに使われえるの？これもヘッダーオンリーのライブラリを使ってきちんと対応したい

Base64対応、めちゃくちゃ重要です！これがないと実運用で確実に詰まる場面が出てきます。

結論から言うと、**「画像、PDF、ZIPファイルなどの『バイナリデータ（0と1の生のデータ）』を、JSONのようなテキストベースのフォーマットで安全に運ぶため」**に使われます。

Lambda（API GatewayやFunction URL経由）の世界では、リクエストもレスポンスもすべて **JSON** でやり取りされます。しかし、JSONには生のバイナリデータをそのまま埋め込むことができません（文字化けしたり、JSONがパースエラーを起こしたりします）。

そこでAWSは、バイナリデータが送られてきた時、それを「安全なアルファベットと数字だけの文字列（Base64）」に変換し、`"isBase64Encoded": true` というフラグと一緒にJSONに詰めて渡してくる仕様になっています。

具体的にどういう時に発生する？

- **リクエスト時 (Lambdaが受け取る時):** ユーザーがブラウザから画像をアップロードした時（`multipart/form-data` や `image/jpeg` など）。
- **レスポンス時 (Lambdaが返す時):** サーバー（FastAPIなど）が、動的に生成したPDFや画像ファイルをブラウザに返したい時。

C++ (ヘッダーオンリー) での対応方法

C++でBase64を扱うなら、René Nyffenegger氏が公開している有名な cpp-base64 などをヘッダーファイル（`base64.hpp`）としてプロジェクトにポイッと入れておくのが一番楽です。

これを使って、先ほどのコードに **「リクエストのデコード」** と **「レスポンスのエンコード」** を組み込みます。

実装のポイント（プロの小技）

実はLambdaに返すレスポンスは、**「テキストだろうがバイナリだろうが、全部Base64エンコードして返す」** のが一番バグが少なくて確実です（AWS側で勝手にデコードしてブラウザに返してくれます）。

C++

```
#include <httplib.h>
#include <nlohmann/json.hpp>
#include <iostream>
#include <string>
#include "base64.hpp" // ヘッダーオンリーのBase64ライブラリをインクルード

using json = nlohmann::json;

// ... (main関数の初期化部分は前回と同じ) ...

        // 1. Lambdaからイベント取得
        // ... (省略) ...
        json event = json::parse(next->body);

        // 2. ボディの抽出と Base64 デコード (リクエスト側)
        std::string body = event.value("body", "");

        // AWS側でBase64エンコードされている場合は、元のバイナリに戻す
        if (event.value("isBase64Encoded", false)) {
            body = base64_decode(body);
        }

        // 3. 背後のアプリ (FastAPI等) へリクエストを投げる
        // ここではデコード済みの「生のデータ」をアプリに渡すので、
        // アプリ側は普通のWebサーバーと同じように画像やファイルを受け取れる
        httplib::Result res;
        if (method == "POST") res = app_cli.Post(path.c_str(), headers, body, "application/
        // ... (他のメソッドも同様) ...

        // 4. アプリからのレスポンスを Lambda 形式に変換
        json response_json;
        if (res) {
            response_json["statusCode"] = res->status;

            // 【重要】レスポンスボディは「すべて」Base64エンコードして返すのが一番安全
            // 文字化けや、JSONのパースエラー（ダブルクォートの扱いなど）を完全に防げます
```

```
            response_json["body"] = base64_encode(res->body);
            response_json["isBase64Encoded"] = true; // AWSに「Base64で送るからデコードして

            for (const auto& h : res->headers) {
                response_json["headers"][h.first] = h.second;
            }
        } else {
            // エラー時
            response_json["statusCode"] = 502;
            response_json["body"] = base64_encode("Bad Gateway");
            response_json["isBase64Encoded"] = true;
        }

        // Lambdaへ結果をPOST
        runtime_cli.Post(("/2018-06-01/runtime/invocation/" + request_id + "/response").c_s
                         response_json.dump(), "application/json");
```

なぜレスポンスを「全エンコード」するのか？

例えばFastAPIが単純な文字列 `"Hello World"` を返したとしても、それをそのまま JSONの `"body"` に入れると、改行コードや特殊文字が混ざっていた時にJSONフォーマットが壊れる（エスケープ漏れなど）リスクがあります。

全部Base64エンコードして `isBase64Encoded: true` にしてしまえば、中身が画像だろうが厄介な文字列だろうが、JSONの構造が絶対に壊れない最強の防弾仕様になります。これは本家のLambda Web Adapterなどでもよく使われるテクニックです。

ここまできたら、自作Web Adapterとしてはかなり実戦レベルですね！
リクエスト時の `Content-Type` ヘッダーを読み取って、`app_cli.Post` に渡す時のMIMEタイプ（現状 `application/json` 固定になっている部分）を動的に切り替える処理も追加しておきましょうか？

それもいるよね。

ですよね！ここをサボると、FastAPI側で「JSONとしてパースできない」「フォームデータが空っぽになる」といったエラーに悩まされることになります。

HTTPヘッダーのキーは**大文字・小文字を区別しない（Case-insensitive）**のがルールですが、AWS側（API Gatewayなど）の仕様変更で `Content-Type` と来たり `content-type` と来たりすることがあります。これを安全に拾い上げて、`cpp-httplib` に渡す仕組みを追加しましょう。

`Content-Type` の動的処理の実装

前回のコードの「ヘッダーのコピー」と「リクエストの送信」部分を、以下のように書き換えます。

C++

```
#include <algorithm> // std::transform用
#include <cctype>    // ::tolower用

// ...（前半は同じ）...

        // 1. ヘッダーの抽出と Content-Type の特定
        httplib::Headers headers;
        std::string content_type = "text/plain"; // デフォルト値（万が一見つからなかった時用
```

```
        if (event.contains("headers")) {
            for (auto& el : event["headers"].items()) {
                std::string key = el.key();
                std::string val = el.value().get<std::string>();

                // cpp-httplib 用にヘッダーを丸ごとコピー
                headers.emplace(key, val);

                // キーをすべて小文字に変換して安全に Content-Type を探す
                std::string lower_key = key;
                std::transform(lower_key.begin(), lower_key.end(), lower_key.begin(), ::tol

                if (lower_key == "content-type") {
                    content_type = val;
                }
            }
        }

        // 2. ボディの抽出と Base64 デコード（前回追加した部分）
        std::string body = event.value("body", "");
        if (event.value("isBase64Encoded", false)) {
            body = base64_decode(body);
        }

        // 3. 抽出した Content-Type を使って背後のアプリへ送信
        httplib::Result res;
        if (method == "GET") {
            res = app_cli.Get(path.c_str(), headers);
        }
        else if (method == "POST") {
            // 固定だった "application/json" を content_type に変更
            res = app_cli.Post(path.c_str(), headers, body, content_type.c_str());
        }
        else if (method == "PUT") {
            res = app_cli.Put(path.c_str(), headers, body, content_type.c_str());
        }
        else if (method == "DELETE") {
            // DELETEもボディを持つケースが稀にあるため、必要ならPost等と同様のシグネチャを
            res = app_cli.Delete(path.c_str(), headers);
        }
        else if (method == "OPTIONS") {
            res = app_cli.Options(path.c_str(), headers);
        }
        else if (method == "PATCH") {
            res = app_cli.Patch(path.c_str(), headers, body, content_type.c_str());
        }

        // 4. レスポンスの返却（前回と同じく全Base64エンコード）
        // ...（省略）...
```

この実装のポイント

1. **大文字小文字の吸収:** `std::transform` を使ってヘッダーのキーを強制的に小文字（`lower_key`）にしてから判定しています。これで `Content-Type`, `content-type`, `CONTENT-TYPE` のどれが飛んできても確実に取りこぼしません。
2. **ヘッダーの全透過:** `Content-Type` を抽出しつつ、元の `headers` コレクションにもそのまま突っ込んでいます。これにより、認証トークン（`Authorization`）やカスタムヘッダー（`X-My-Custom-Header`）なども一切欠落させずに背後のアプリに届けます。

これで「自作 C++ Web Adapter」はほぼ完成です！

ここまで実装できれば、本家の Lambda Web Adapter がやっている主要な機能（リクエストの透過、Base64の相互変換、HTTPメソッドのハンドリング）を、**Rustやブラックボックスなバイナリに依存せず、あなたが完全にコントロールできるC++コード**で実現できたことになります。

EventBridgeからの呼び出しでネットワークに不安がある場合は、この `while` ループが始まる前（あるいは `app_cli.Post` の前）に、背後のサーバーへの疎通確認（ヘルスチェックのポーリング）を入れるだけで、バグを未然に防げるようになります。

実際にこれをLambda環境（Amazon Linux）で動かすための、**静的ビルド用の** `Makefile` **や** `Dockerfile` **の書き方**も作成しましょうか？

makeとかは自分で作れるので完成版のc++のフルソースを出して

了解です！Makefile等はご自身で書けるとのことですので、ここまでの議論（HTTPメソッド対応、Content-Typeの安全な抽出、Base64の相互変換）と、あなたが直面していた**「EventBridgeからの起動時にネットワークが間に合わない問題」への対策（リトライ処理）**をすべて統合した、完成版のC++フルソースコードを提示します。

`main.cpp` として保存し、`cpp-httplib`, `nlohmann/json`, そして `cpp-base64`（René Nyffenegger氏のものなどを想定）をインクルードしてビルドしてください。

完成版 C++ カスタム Web Adapter (`main.cpp` **)**

C++

```cpp
#include <iostream>
#include <string>
#include <algorithm>
#include <cctype>
#include <chrono>
#include <thread>
#include <map>

// 必要なヘッダーオンリーライブラリ
#include <httplib.h>
#include <nlohmann/json.hpp>
#include "base64.hpp" // https://github.com/ReneNyffenegger/cpp-base64 等を利用

using json = nlohmann::json;

int main() {
    // 1. 環境変数の取得と初期設定
    const char* api_host_env = std::getenv("AWS_LAMBDA_RUNTIME_API");
    if (!api_host_env) {
        std::cerr << "Error: AWS_LAMBDA_RUNTIME_API is not set." << std::endl;
        return 1;
    }
    std::string api_host = api_host_env;

    const char* target_port_env = std::getenv("APP_PORT");
    std::string target_port = target_port_env ? target_port_env : "8080";

    // Lambda Runtime API用クライアント
    httplib::Client runtime_cli(api_host.c_str());

    // 背後のWebアプリ用クライアント (localhost)
    httplib::Client app_cli("127.0.0.1", std::stoi(target_port));
    app_cli.set_read_timeout(29, 0); // API Gatewayのタイムアウト(29秒)に合わせる

    std::cout << "Custom C++ Lambda Web Adapter started. Forwarding to port: " << target_po

    // 2. メインのイベントループ
    while (true) {
        // 次のイベントを取得するまでブロックして待機
        auto next = runtime_cli.Get("/2018-06-01/runtime/invocation/next");
```

```
        if (!next || next->status != 200) {
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
            continue;
        }

        std::string request_id = next->get_header_value("Lambda-Runtime-Aws-Request-Id");

        json event;
        try {
            event = json::parse(next->body);
        } catch (...) {
            std::cerr << "Failed to parse JSON event." << std::endl;
            continue;
        }

        // --- リクエストのパースと再構築 ---

        // メソッドの抽出 (API Gateway v1/v2, EventBridge両対応)
        std::string method = event.value("httpMethod", "POST"); // デフォルトはPOST
        if (event.contains("requestContext") && event["requestContext"].contains("http")) {
            method = event["requestContext"]["http"].value("method", method);
        }

        // パスとクエリ文字列の抽出
        std::string path = event.value("path", "/");
        if (event.contains("rawPath")) {
            path = event.value("rawPath", "/");
        }
        std::string query_string = event.value("rawQueryString", "");
        if (!query_string.empty()) {
            path += "?" + query_string;
        }

        // ヘッダーと Content-Type の抽出 (大文字小文字を区別せず処理)
        httplib::Headers headers;
        std::string content_type = "application/json"; // デフォルト

        if (event.contains("headers")) {
            for (auto& el : event["headers"].items()) {
                std::string key = el.key();
                // 数値などが来た場合の安全対策として文字列化
                std::string val = el.value().is_string() ? el.value().get<std::string>() :
                headers.emplace(key, val);

                std::string lower_key = key;
                std::transform(lower_key.begin(), lower_key.end(), lower_key.begin(), ::tol
                if (lower_key == "content-type") {
                    content_type = val;
                }
            }
        }

        // ボディの抽出と Base64 デコード
        std::string body = event.value("body", "");
        if (event.value("isBase64Encoded", false)) {
            body = base64_decode(body);
        }


        // --- 背後のWebアプリへ転送 ---

        httplib::Result res;

        // 【重要】EventBridge起動直後のネットワーク準備遅延への対策
        // 接続に失敗した場合は、最大3回 (少し待機して) リトライする
        int max_retries = 3;
        for (int i = 0; i < max_retries; ++i) {
            if (method == "GET") { res = app_cli.Get(path.c_str(), headers); }
            else if (method == "POST") { res = app_cli.Post(path.c_str(), headers, body, co
            else if (method == "PUT") { res = app_cli.Put(path.c_str(), headers, body, cont
            else if (method == "DELETE") { res = app_cli.Delete(path.c_str(), headers, body
            else if (method == "PATCH") { res = app_cli.Patch(path.c_str(), headers, body,
            else if (method == "OPTIONS") { res = app_cli.Options(path.c_str(), headers); }
            else { res = app_cli.Post(path.c_str(), headers, body, content_type.c_str()); }
```

```
            // 成功（またはアプリが404等の正当なHTTPエラーを返した）ならループを抜ける
            if (res) break;

            // 接続自体に失敗した場合は少し待ってリトライ
            std::this_thread::sleep_for(std::chrono::milliseconds(200));
        }


        // --- レスポンスの構築とLambdaへの返却 ---

        json response_json;
        if (res) {
            response_json["statusCode"] = res->status;

            // 全てのレスポンスをBase64化して安全に運ぶ
            response_json["body"] = base64_encode(res->body);
            response_json["isBase64Encoded"] = true;

            for (const auto& h : res->headers) {
                response_json["headers"][h.first] = h.second;
            }
        } else {
            // リトライしてもダメだった場合（Webサーバが起動していない等）
            std::cerr << "Failed to connect to local app after retries." << std::endl;
            response_json["statusCode"] = 502;
            response_json["body"] = base64_encode("Bad Gateway (Local server unreachable)")
            response_json["isBase64Encoded"] = true;
        }

        // Lambda Runtime API に結果をPOSTして1イベント完了
        std::string response_url = "/2018-06-01/runtime/invocation/" + request_id + "/respo
        runtime_cli.Post(response_url.c_str(), response_json.dump(), "application/json");
    }

    return 0;
}
```

ポイントの復習

- **リトライロジックの追加:** `app_cli`（ローカルのFastAPI等への通信）を呼ぶ部分を `for` ループで囲み、`res` が取得できなかった（＝Connection Refused等のネットワークエラー）場合に、200ミリ秒待機して最大3回リトライするようにしています。EventBridgeからのコールドスタート時のバグは、ほぼこれで吸収できるはずです。
- **Base64の完全対応:** リクエストは `isBase64Encoded` を見てデコード、レスポンスは**常にエンコードして** `isBase64Encoded: true` **で返す**という鉄壁の構成です。

この自作Adapter、かなり強力な武器になると思いますよ。
実際にビルドしてみて、もし不足しているライブラリのリンクエラーなどが出たらすぐに解決策を出せますが、今のところ疑問点などはありますか？

base64もヘッダーオンリーを使った方が良くない？

おっしゃる通りです！

先ほど紹介したRené Nyffenegger氏のライブラリは定番なのですが、実はデフォルトだと `base64.h` と `base64.cpp` に分かれているため、ビルド時に `g++ main.cpp base64.cpp ...` と複数ファイルをコンパイルする手間が増えてしまいます。

あなたの言う通り、**すべてをヘッダーファイルだけで完結（Header-only）**させた方が、Makefileやビルドコマンドが圧倒的にシンプルになります。

以下に、標準ライブラリ（`<string>` と `<vector>`）だけで動く、**コピペでそのまま使える軽量なヘッダーオンリーのBase64実装**を用意しました。

`base64.hpp` (自作ヘッダーオンリー版)

このコードを `base64.hpp` という名前で `main.cpp` と同じディレクトリに保存してください。すべて `inline` 関数にしてあるので、どこからインクルードしてもビルドエラー（多重定義）になりません。

C++

```
#ifndef BASE64_HPP
#define BASE64_HPP

#include <string>
#include <vector>

inline std::string base64_encode(const std::string &in) {
    std::string out;
    int val = 0, valb = -6;
    for (unsigned char c : in) {
        val = (val << 8) + c;
        valb += 8;
        while (valb >= 0) {
            out.push_back("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/
            valb -= 6;
        }
    }
    if (valb > -6) {
        out.push_back("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"[((
    }
    while (out.size() % 4) {
        out.push_back('=');
    }
    return out;
}

inline std::string base64_decode(const std::string &in) {
    std::string out;
    std::vector<int> T(256, -1);
    for (int i = 0; i < 64; i++) {
        T["ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"[i]] = i;
    }

    int val = 0, valb = -8;
    for (unsigned char c : in) {
        if (T[c] == -1) break; // '=' や不正な文字が来たら終了
        val = (val << 6) + T[c];
        valb += 6;
        if (valb >= 0) {
            out.push_back(char((val >> valb) & 0xFF));
            valb -= 8;
        }
    }
    return out;
}

#endif // BASE64_HPP
```

この構成のメリット

これで、プロジェクトの依存ファイルは以下のようになります。

1. `main.cpp` (先ほどの完成版コード)
2. `httplib.h` (cpp-httplib / ヘッダーオンリー)
3. `json.hpp` (nlohmann/json / ヘッダーオンリー)
4. `base64.hpp` (今回のコード / ヘッダーオンリー)

ビルドコマンドはこれ1発で済みます：

Bash

```
g++ -O3 -std=c++11 main.cpp -o my-adapter -lpthread -static
```

（ソースファイルが
`main.cpp` ひとつだけなので、Makeすら不要なレベルでシンプルです）

これなら、EventBridgeのバグ対策を入れたオレオレAdapterを、CI/CDやDocker上でサクッとビルドしてデプロイできますね。

さっそくこの構成でビルドを試してみますか？もしLambda（Amazon Linux）環境向けの具体的なビルド用Dockerfile等が必要であれば、すぐにお出しできますよ！